One P-Bit per Parameter: Task Quality, Failure Mechanisms, and Resource Tradeoffs in Pretrained Neural Networks

Omar Ramadan
Independent Research

November 6, 2026
Public-access revision: November 8, 2026

Abstract

A programmable probabilistic bit can represent a scalar through its stationary probability, exchanging spatial replication for temporal estimation. The principle is established, but its usefulness for converting pretrained neural networks depends on finite controller precision, local ranges, layer sensitivity, and sampling cost. We investigate an affine single-p-bit encoding with 12-bit probability commands and shared range metadata. Complete five-seed evaluations convert every learned scalar of GPT-2, DistilBERT, and LFM3-1.7B, including normalization and bias parameters. At 32,768 independent samples per scalar, selected layouts yield perplexity ratios of 1.00659 and 1.00259 for the two language models and mean classification accuracy of 87.4944% versus 87.512% for DistilBERT. All satisfy their frozen task-quality margins on the evaluated benchmarks. A controlled GPT-2 study identifies the output head and range orientation as a failure mechanism: the original layout fails at the same sample budget. Sensitivity-based allocation reduces nominal reconstruction observations by 41.7% while preserving the GPT-2 criterion. Native THRML operator tests, variance-matched controls, and layerwise interventions qualify these results. Stronger four-element controls can achieve lower operator error, and a finite-controller collision excludes universal lossless BF16 equivalence for the implemented format. The results establish scoped software feasibility and diagnostic methods, not a one-bit model file or a physical energy advantage.

1 Introduction

A binary state contains one bit at an instant, whereas the distribution of a programmable stochastic binary element can encode a continuous quantity. This distinction motivates single-p-bit weight encoding (SPW): assign one logical programmable probabilistic element to each learned scalar and estimate its value over time. For weights restricted to $[0, 1]$, the simplest assignment is $p = w$. Signed weights require a shared affine range.

Encoding values through Bernoulli streams is established stochastic computing, including early neural implementations [12]. The research question is therefore operational: can pretrained networks tolerate a finite, resource-accounted realization of this representation, and what causes success or failure? An instantaneous binary sample is not BF16. A programmable probability also requires a controller, whose precision and storage cannot be omitted from a compression claim.

We study this question using local NVIDIA DGX Spark/GB10 execution, a reproducible finite-controller simulator, and native operator experiments with Extropic's THRML library [2]. The main neural experiments reconstruct a weight tensor from sampled counts and then execute ordinary

BF16 inference. This mode permits controlled analysis at model scale without claiming a full hardware mapping.

The empirical contribution has four parts. First, we evaluate complete stochastic cohorts for three pretrained architectures, including a 1.73-billion-parameter hybrid model. Second, we identify a concrete GPT-2 failure mechanism through output-head interventions and activation-weighted variance. Third, we validate a reduced sampling allocation without relaxing the one-logical-p-bit assignment. Fourth, we expose limitations using a stronger spatial control, calibration perturbations, and a finite-format counterexample. The accompanying supplement records implementation details, historical outcomes, and the evidence map. The study is an investigation of tested regimes; it does not establish universal architecture or task transfer.

2 Related work and positioning

Stochastic representations. Bit-stream neural networks already represent values with Bernoulli sequences [12]. Probabilistic binary networks use stochastic weight distributions and can form ensembles [9]. Progressive stochastic binarization also controls approximation quality by sampling, including adaptive allocation [4]. Consequently, neither probability encoding nor trading time for precision is a novelty claim here. Our focus is finite-control conversion of existing pretrained checkpoints, complete-parameter coverage, and diagnosed cross-operator failure.

Post-training quantization. GPTQ uses post-training reconstruction for transformer quantization [3]; AWQ uses activation information to protect important weight directions [5]. Our activation-weighted selection is related in motivation, but its objective is the variance of a stochastic affine representation and its cost includes temporal samples. We have not completed a matched end-to-end comparison with optimized GPTQ or AWQ deployment. The present results should not be interpreted as superiority over contemporary low-bit inference.

Probabilistic hardware. Extropic's Z1T work studies sparse transformer-like computation with multi-p-bit representations and temporally accumulated statistics [1]. We use the official THRML sampler for small native experiments, but do not reproduce the complete Z1T system. Our four-element controls are explicitly defined internal baselines. Native operator validation, ideal independent-count simulation, and physical hardware measurement are distinct evidence levels throughout this paper.

3 Representation and fundamental limits

3.1 Affine stationary-probability encoding

For parameter i in group $g(i)$, let c_g be the center and $r_g \geq 0$ the radius. The ideal representation is

$$w_i = c_g + r_g(2p_i - 1), \qquad z_i^{(t)} \sim \text{Bernoulli}(p_i). \tag{1}$$

With independent samples, the temporal estimate is

$$\hat{w}_{i,T} = c_g + r_g \left(2\frac{1}{T}\sum_{t=1}^{T} z_i^{(t)} - 1 \right), \quad \text{Var}(\hat{w}_{i,T}) = \frac{4r_g^2 p_i(1-p_i)}{T}. \tag{2}$$

For $r_g = 1$, the worst-case standard deviation is $T^{-1/2}$; the rough metric $b_{\text{eff}} = -\log_2 \sigma$ grows as $\frac{1}{2}\log_2 T$. This is an estimation-error metric, not a floating-point format, mantissa count, or universal BF16 equivalence criterion.

3.2 Implemented finite controller

The implementation stores the group minimum l_g and span $d_g = u_g - l_g$ as FP32 values. This is algebraically equivalent to $c_g = l_g + d_g/2$ and $r_g = d_g/2$, up to floating-point evaluation. For a nonconstant group,

$$p_i^* = (w_i - l_g)/d_g, \qquad \epsilon = \text{sigmoid}(-8), \tag{3}$$

$$h_i^* = \tfrac{1}{2}\,\text{logit}(\text{clip}(p_i^*, \epsilon, 1-\epsilon)), \tag{4}$$

$$q_i = \text{round}\left[\frac{4095}{8}(h_i^* + 4)\right], \qquad q_i \in \{0, \ldots, 4095\}, \tag{5}$$

$$\tilde{p}_i = \text{sigmoid}(2\hat{h}_i), \qquad \hat{h}_i \simeq 8q_i/4095 - 4. \tag{6}$$

The deployed mapping evaluates the decoded field in FP32 before the sigmoid calculation in FP64; the exact operation order is fixed in the released source. Constant groups use $p_i^* = 1/2$ and $d_g = 0$, so their reconstructed value is invariant to samples. Finite bias means group endpoints generally have small systematic error. More samples reduce variance around $l_g + d_g\tilde{p}_i$, not this controller bias.

For a deterministic target and independent samples,

$$\text{MSE}(\hat{w}_{i,T}, w_i) = (l_g + d_g\tilde{p}_i - w_i)^2 + \frac{d_g^2\tilde{p}_i(1-\tilde{p}_i)}{T}. \tag{7}$$

Neither quantized controls nor a shared range constitute an additional stochastic element. They are nevertheless additional stored information.

3.3 Three execution regimes

We distinguish **R1**, reconstructing a sampled matrix then using deterministic neural operators; **R2**, computing each linear operator with sampled weights and accumulating outputs; and **R3**, reformulating computation around expectations. The complete model results use R1. Independent count simulation draws $n_i \sim \text{Binomial}(T, \tilde{p}_i)$ and computes $l_g + d_g n_i/T$. This has the same marginal law as summing T independent Bernoulli observations, without explicitly generating those observations.

For fixed inputs and identical weight samples, linearity gives

$$X\left(\frac{1}{T}\sum_t W^{(t)}\right) = \frac{1}{T}\sum_t XW^{(t)}. \tag{8}$$

Native tests verify this identity numerically. It does not supply an inherent variance advantage to R2, and does not extend to moving averaging through nonlinearities or to averaging complete stochastic networks. R1's sampled matrix is reused across the dataset, so examples share the same weight draw.

3.4 Why universal BF16 equality is excluded

An executed counterexample gives two distinct BF16 values, 2^{-20} and 2^{-19} in the tested $[0, 1]$ range, that map to identical finite programs. No additional samples can distinguish identical programs.

More generally, for fixed groups of 128, the control and range fields allow at most $2^{12.5N}$ programs. Even counting N additional persistent binary states gives $2^{13.5N}$, below the $65{,}280^N$ possible finite BF16 bit-pattern tensors. This counting argument is conditional on the stated format and fixed grouping; extra metadata must be counted if it conveys additional information. It rules out universal lossless conversion for this format, not useful model-specific approximation.

4 Experimental methods

4.1 Models, data, and numerical execution

Table 1 specifies the full evaluations. GPT-2 [10] and LFM3 [6] use WikiText-2 [8], with their own tokenizers and fixed article/window layouts. DistilBERT [11] uses the SST-2-finetuned checkpoint evaluated on IMDb [7]; its BF16 reference therefore includes the same task transfer. Every learned scalar is converted, including biases and normalization parameters. Tied input embeddings and output heads share one parameter representation and are counted once.

Table 1: Complete evaluated checkpoints and task coverage. Token counts are scored targets, not the number of raw words.

Model	Unique parameters	Scored cases	Task
GPT-2	124,439,808	283,225 tokens	WikiText-2
DistilBERT	66,955,010	25,000 reviews	IMDb classification
LFM3-1.7B	1,728,801,536	294,442 tokens	WikiText-2

Model-scale inference uses BF16 weights and neural execution, with explicitly recorded loss accumulation and deterministic settings. GPT-2 has 586 scored windows across 62 articles; LFM3 has 607 windows across the same number of articles. DistilBERT truncates inputs at 256 tokens and evaluates batches of 32. LFM3 uses CPU master weights, leaf-module GPU offloading, and the original full-sequence output head; no KV cache is used. Research GPU jobs run serially with a 2 GiB PyTorch allocator cap on the shared GB10. No cloud compute is used in the final Spark cohorts.

4.2 Selection and statistical protocol

Training activations determine range orientation separately for each operator. Matrix groups contain 128 consecutive entries after the selected permutation. LFM3 uses groups of seven for depthwise convolution filters and sixteen for normalization tensors. The uniform condition uses $T = 32{,}768$ with 12-bit controls. Hyperparameters and seed lists are fixed before the corresponding complete evaluation. The benchmarks had been inspected in earlier experiments; these are prospective evaluations of fixed policies on previously seen benchmark datasets, not pristine unseen-data tests.

Let ℓ_0 be the mean BF16 negative log likelihood and ℓ_s the value for independent weight draw s. For five seeds, $d_s = \ell_s - \ell_0$ and

$$R = \exp(\bar{d}), \qquad I_{95} = \exp\left[\bar{d} \pm 2.776445\frac{s_d}{\sqrt{5}}\right]. \tag{9}$$

A condition passes the frozen probability-quality gate if the upper endpoint is at most 1.02. This is a conditional Monte Carlo interval on the fixed evaluation data. DistilBERT additionally requires the upper 95% bound on absolute accuracy decline to be at most two percentage points. Its exponentiated

classification cross-entropy is termed categorical perplexity and is not a language-model perplexity. We retain all seeds; there is no favorable-seed stopping.

Saved per-example or per-token arrays are independently rescored. A separate SDK reconstructs all evaluated weight points and replays prespecified neural windows. These replays verify implementation reproducibility, not physical device fidelity. Article/seed bootstrap intervals are supplementary and do not replace the frozen primary gate.

5 Complete model results

Table 2: Five-seed complete-model evaluations at $T = 32{,}768$. Perplexities for stochastic language models are exponentials of mean seed NLL. Brackets give conditional Monte Carlo 95% t_4 intervals for the ratio.

Condition	BF16 PPL	K1 PPL	PPL ratio [95% interval]
GPT-2, original layout	25.50149	27.02231	1.05964 [1.04254, 1.07701]
GPT-2, selected layout	25.50149	25.66953	1.00659 [0.99927, 1.01397]
LFM3, selected layout	18.93015	18.97918	1.00259 [0.98751, 1.01790]

Both selected language-model conditions pass the predefined 2% upper-bound criterion (Table 2). DistilBERT's mean accuracy is 87.4944%, compared with 87.512% for BF16; the upper interval endpoint for accuracy decline is 0.08383 percentage points. Its categorical-perplexity ratio is 0.99966 [0.99676, 1.00256], so both criteria pass. Figure 1 also shows that original-layout GPT-2 fails at the same sample count. Model size alone therefore does not explain acceptance: layout and operator sensitivity matter.

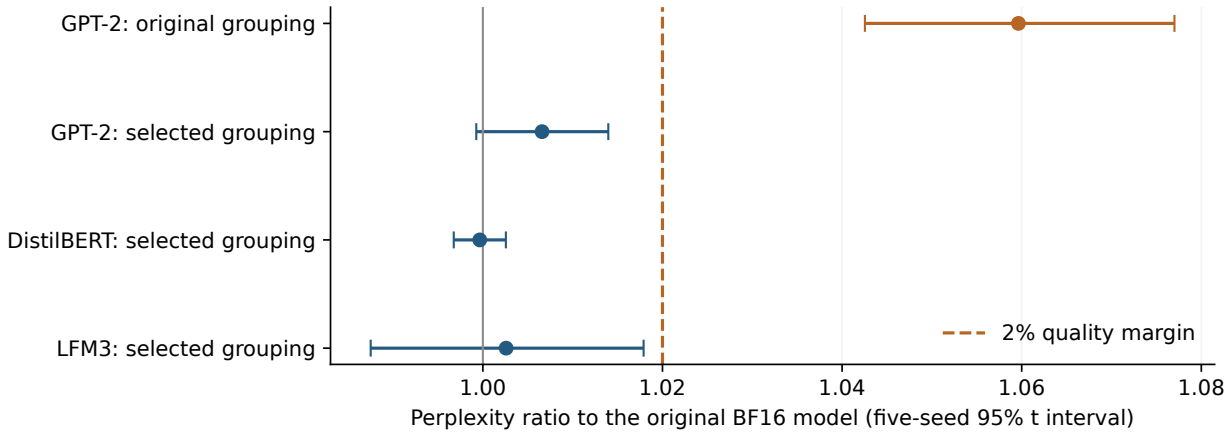


Figure 1: Complete-task quality relative to the original BF16 references. Bars are five-seed conditional Monte Carlo 95% intervals, not cross-task uncertainty. DistilBERT uses categorical perplexity. All selected policies pass; original-layout GPT-2 fails.

The finite-controller mean condition helps separate systematic and stochastic error. GPT-2's mean-condition PPL is 25.47279; LFM3's is 18.95199. Neither the mean condition nor stochastic PPL below the reference should be interpreted as improved generalization: the intervals and limited task coverage do not support that claim. LFM3's joint seed/article bootstrap ratio interval is [0.99188, 1.01065], reported as a supplementary uncertainty calculation.

All twelve GPT-2 and twelve DistilBERT weight points were independently reconstructed, with 36 neural windows/batches replayed for each architecture. LFM3 replay checked all seven model points, 1,430 stochastic parameter-count hashes, and 21 full-network windows. All prescribed comparisons matched bitwise on the recorded local runtime. This coverage is substantial but is not a replay of every neural window.

6 Why some configurations fail

6.1 Output-head interventions in GPT-2

The original flat grouping exposes a sensitive output-head direction. On a fixed training diagnostic, BF16 PPL is 45.7987 and the original-layout stochastic condition is 58.9795. Restoring only the input-embedding use of the tied matrix gives 59.6616, whereas restoring only the output-head use gives 46.1026. Restoring both gives 46.2131. Reorienting the tied matrix, while retaining stochastic representation, gives 45.2709 in this draw (Figure 2).

These role-separated restorations use hooks to intervene on one use of a shared parameter. They are diagnostic controls, not implementable all-K1 proposals with two contradictory versions of one tied matrix. The final layout proposal uses a single shared representation.

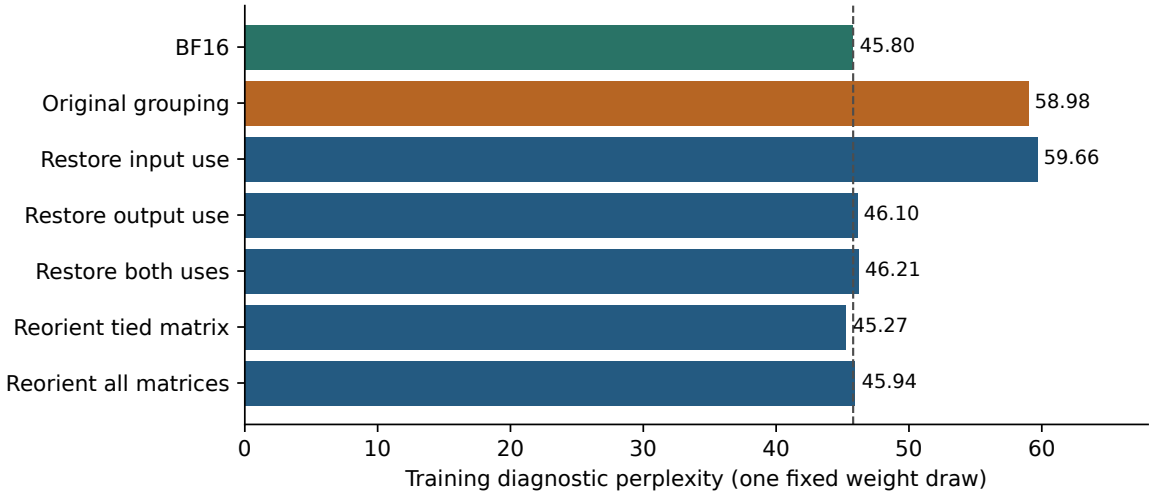


Figure 2: GPT-2 training-only causal interventions on one fixed stochastic draw. Restoring the output use largely removes the failure; restoring the input use does not. These bars have no population confidence intervals and are not the full-benchmark scores in Table 2.

6.2 Activation-weighted range orientation

For a fixed input x and independently sampled weights, the variance of output coordinate j is

$$\mathrm{Var}(\hat{y}_j \mid x) = \sum_i x_i^2 \frac{d_{ji}^2 \tilde{p}_{ji}(1-\tilde{p}_{ji})}{T}. \tag{10}$$

Thus equal average weight error does not imply equal output error. A large hidden feature can magnify error in its corresponding output-head column. Grouping changes the local span and can concentrate or suppress precisely those directions. With clean hidden activations fixed, the

GPT-2 head's predicted output-logit variance falls from 0.61444 to 0.002698 after reorientation, approximately 228-fold. This is a conditional operator-variance reduction, not a 228-fold model speedup.

The policy must be selected per operator. Reorienting one examined DistilBERT feed-forward matrix instead increases predicted variance by approximately 3.2 times. The cross-model result supports measurement-driven layouts, rather than a universal instruction to transpose every matrix. The matched full GPT-2 cohorts provide stronger causal evidence than the single-draw diagnostic alone: changing layouts at fixed T changes failure to acceptance.

6.3 Layer, channel, and nonlinear propagation

The investigation traces all twelve GPT-2 blocks and 144 attention heads, and all 24 LFM3 blocks with 2,048 hidden channels. LFM3 combines convolutional and attention components; its 55-condition study contains the baseline, full stochastic model, 24 isolated-block substitutions, 24 block restorations, and five component-family restorations. Every isolated substitution leaves preceding blocks exactly unchanged.

The most harmful isolated block in the fixed LFM3 draw is block 3, whereas restoring block 2 gives the largest improvement in the fully stochastic network. This mismatch shows that isolated effects do not simply add. The corresponding family restorations include embeddings, normalization, attention, convolution, and feed-forward components. These mixed deterministic/stochastic conditions are diagnostic relaxations of the all-K1 constraint.

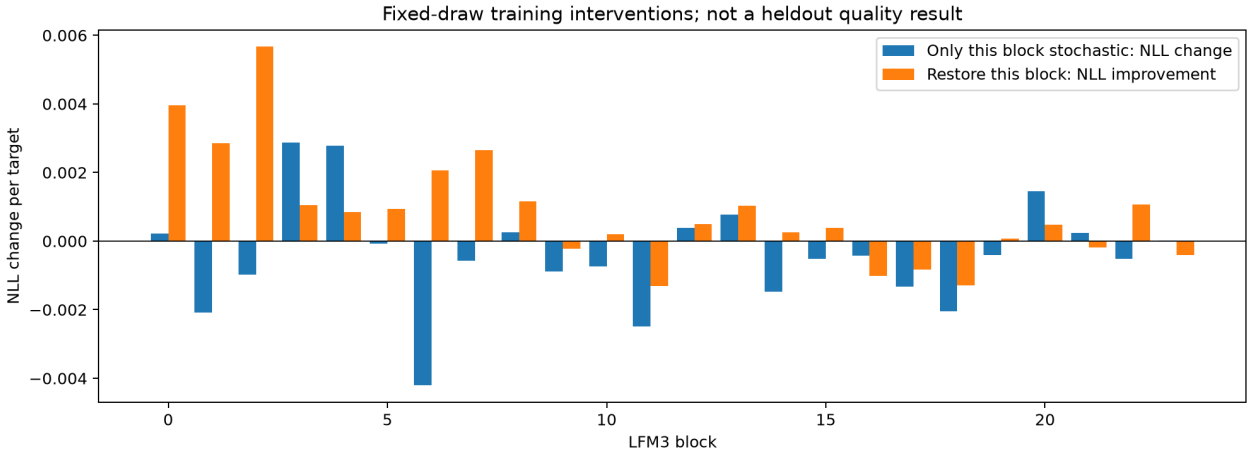


Figure 3: LFM3 isolated-block and restoration diagnostics on 1,024 training targets and one fixed weight draw. Layer numbers are zero-based. The interventions distinguish injection sensitivity from effects in the already-perturbed network; they are not independent test-quality gains.

Error magnitude must also be separated from signal scale. Between LFM3 blocks 11 and 12, error power increases 20.76% while signal power increases 57.03%; relative RMS error consequently falls 12.305%. This is not evidence of denoising. Some early injected errors grow strongly in absolute size downstream, but these finite operating-point measurements are not universal amplification constants.

6.4 Conditional error covariance

For each fixed input position, we compute sample covariance across five independently reconstructed LFM3 models, then average covariance matrices over 8,224 positions. Two 64-channel subsets per

block are examined: channels selected for high error on the same cohort and a fixed random subset. All 48 matrices are checked against saved sufficient statistics and the original trace diagonals.

Across blocks, mean absolute off-diagonal correlation ranges from 0.0246 to 0.1066 in the selected subset and from 0.0166 to 0.0327 in the random subset. In the last block, the selected subset has covariance participation ratio 4.50 and leading-eigenvalue fraction 0.460, versus participation ratio 56.55 for the random subset. Independent weight sampling can therefore induce correlated output errors through shared computation. These are descriptive subset statistics from training inputs, not full 2,048-channel covariance estimates or population rank claims.

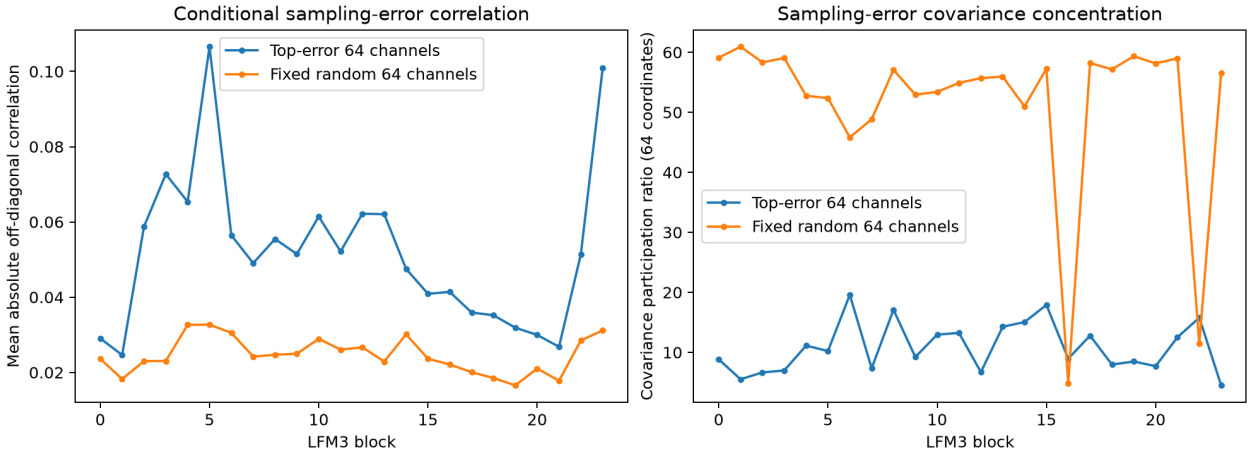


Figure 4: LFM3 conditional output-error covariance summaries. Covariance is computed across five draws at each fixed input position and averaged over positions. Selected-channel results use the same cohort for selection; differences from the random subset are descriptive.

7 Reducing temporal sampling cost

A first-order loss-fluctuation proxy weights per-parameter variance by squared loss gradients. If group g has coefficient A_g and N_g parameters, the continuous relaxation

$$\min_{T_g>0} \sum_g N_g T_g \quad \text{subject to} \quad \sum_g A_g/T_g \le V \tag{11}$$

allocates T_g proportional to $\sqrt{A_g/N_g}$ before bounds and rounding. This is a heuristic for finite neural inference, not a theorem about test loss. The executed policy selects directions and sample counts from training/calibration data and uses groups of sixteen for small normalization and bias vectors. It does not fine-tune model weights.

A fresh five-seed GPT-2 cohort validates the resulting policy on all 283,225 targets. Nominal observations per reconstruction decrease from 4,077,643,628,544 to 2,378,245,865,472, a reduction of 41.676%. Additional range metadata is 53,088 bytes. The maximum sample count remains 32,768 and all 124,439,808 parameters retain K1 assignment. The PPL ratio is 1.00780 [0.99907, 1.01660], passing the same margin (Figure 5).

All six model points and eighteen prescribed neural windows replay independently. Uniform and reduced policies use separate seed cohorts; passing the same margin does not prove their accuracies identical. The cost reduction counts nominal Bernoulli observations for R1 reconstruction. It is neither a measured GPU speedup nor a measured reduction in inference energy.

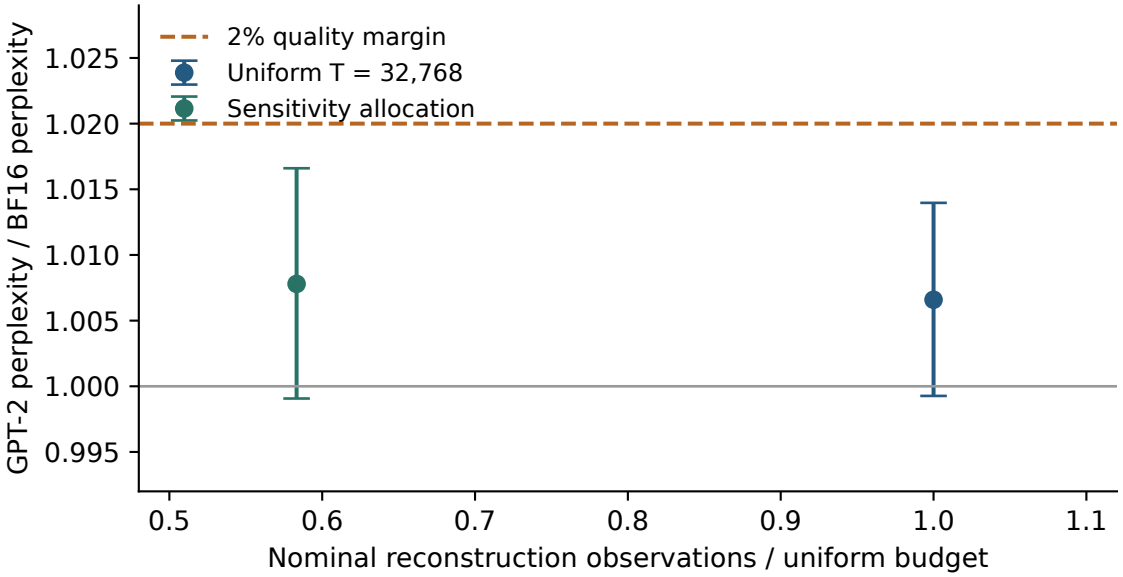


Figure 5: Validated sensitivity-based allocation reduces nominal reconstruction work to 58.324% of the uniform budget. Vertical bars show independent five-seed Monte Carlo intervals. The sampled matrix is reused across inputs.

8 Native controls and device nonidealities

8.1 A stronger spatial baseline

The native THRML comparison uses a real 8×768 GPT-2 output-head bank and eight fixed hidden vectors. A strong K4 baseline represents normalized u by three clamped coarse bits $q = \min(\lfloor 8u \rfloor, 7)$ and one probabilistic fractional bit with target $f = 8u - q$, decoded as $(q + z)/8$. Its fractional command uses the same 12-bit finite field grid. This is an internal baseline, not a claim about Extropic's production encoder.

Table 3: Native THRML reoriented-head control, five draws. Scheduled observations count all nodes; free updates count only stochastic nodes. Output MSE is averaged over the fixed operator inputs.

Condition	Elements	T	All-node observations	Mean MSE
K1	6,144	32,768	201,326,592	7.0041×10^{-5}
K4 coarse/fractional	24,576	8,192	201,326,592	3.3115×10^{-6}
K4 coarse/fractional	24,576	32,768	805,306,368	5.6909×10^{-7}

The stronger K4 condition has lower operator error at matched scheduled observation count (Table 3). It uses more elements and additional coarse information, so K1 is not generally accuracy-dominant at equal accounting. Free-node update counts for the three rows are respectively 201,326,592, 50,331,648, and 201,326,592. Clamped-node reads, holding, and programming must be distinguished from stochastic transitions. Including controls and instantaneous states gives 13 bits per K1 scalar versus 19 per K4 scalar before shared ranges, under this accounting convention.

All thirty native cases are audited; three full trajectories replay bitwise. Exact binomial calculations include cross-input covariance and fourth moments. A sixteen-outcome enumeration checks the moment formula. Small-cohort empirical means need not equal analytical expectations

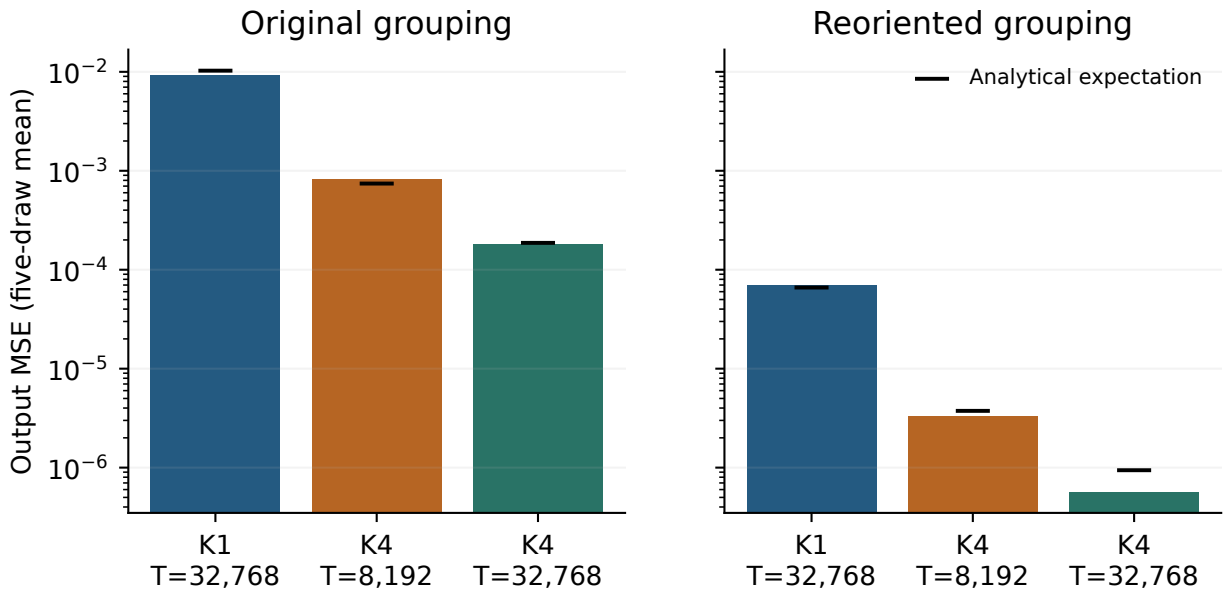


Figure 6: Native THRML operator comparison under both layouts. Bars show empirical five-draw mean MSE; black marks show analytical expectations. These are not confidence intervals. Shared sampled weights correlate errors across the eight input vectors, which is included in the independent moment audit.

exactly.

8.2 Variance-matched and calibration controls

A full GPT-2 training diagnostic matches Gaussian noise to each finite-controller mean and estimator variance before numerical rounding. Across five draws, Bernoulli reconstruction gives a PPL ratio of 0.99688 [0.98409, 1.00984], and Gaussian noise gives 1.00406 [0.98855, 1.01981]. These results do not establish a Bernoulli-specific quality advantage or distributional equivalence.

A separate illustrative device model uses

$$p_i^{\mathrm{dev}} = \mathrm{sigmoid}\{a_i \,\mathrm{logit}(\tilde{p}_i) + b_i\}, \quad a_i \sim \mathcal{N}(1, 0.02^2), \quad b_i \sim \mathcal{N}(0, 0.02^2). \tag{12}$$

The uncalibrated-to-ideal paired PPL ratio is 1.03229 [1.00038, 1.06522] on 1,024 training targets. An oracle knowing the true gain and offset reduces mean squared bias by about 447-fold, but retains clipping and finite control quantization. Storing two FP32 calibration values per GPT-2 scalar would cost an additional 995,518,464 bytes. This oracle is a bound on achievable correction, not a measured calibration procedure. Separate operator-level moment calculations and probe simulations examine correlation, drift, and prior-aware calibration; their scopes and costs are documented in the supplement.

9 Resource accounting and implementation

The K1 label counts stochastic elements; it does not count all information required to program them. For N parameters and G range groups, ideal commands and FP32 ranges require $12N + 64G$ bits. Counting one instantaneous state per parameter gives $13N + 64G$ bits. The implementation uses uint16 command arrays, requiring $16N + 64G$ bits before conceptual states. For groups of

128, commands, ranges, and states amount to 13.5 ideal bits per parameter, or 17.5 using uint16 commands. Decoded BF16 working weights and framework/transient memory are additional.

For LFM3 the ideal packed command-and-range payload is 2,701,541,888 bytes, versus 3,565,942,656 bytes with uint16 commands. R1 additionally holds reconstructed BF16 weights totaling 3,457,603,072 bytes; retaining an original reference duplicates that payload. Maximum recorded GPU allocation is 824,188,928 bytes, which reflects offloading and is not total CPU/GPU process memory.

A hardware energy model would need switching, bias programming, retention, readout, accumulation, and recalibration costs, as well as device capacity and correlation-adjusted sampling rate. If programming is reused over M inferences, a schematic cost is

$$E_{\text{infer}} = E_{\text{transitions}} + E_{\text{readout}} + E_{\text{accumulation}} + \frac{E_{\text{program}}}{M} + E_{\text{calibration}} + E_{\text{retention}}. \tag{13}$$

These terms are not identified by the present GPU experiments. R1 can amortize reconstruction across inputs; hardware-native R2 has a different schedule. Neither node counts nor binomial simulator time alone proves an energy or latency advantage.

The public PBit1 SDK provides the finite-controller reconstruction API and an optional pinned THRML backend. The tested development distribution passes 23 installed-package tests, with zero skips, and a native CLI smoke test. The public research repository contains source snapshots, protocols, compact evidence, figure-generation code, and build instructions. Large raw arrays and checkpoints remain separate, with hashes and restoration requirements. Their presence in local validation must not be confused with a self-contained public data release.

10 Limitations and conclusion

The study covers three checkpoint/task evaluations on two previously inspected tasks. It does not establish instruction-following quality, long-context generation, robustness under distribution shift, or transfer to every architecture. Five-seed intervals quantify sampling variability on fixed texts; the causal layer rankings use a single draw. No full-model native THRML execution or physical device measurement is reported. Probability-aware fine-tuning was unnecessary for the accepted frozen-model criteria and is not claimed as a tested universal remedy. A competitive matched-quality comparison against optimized low-bit inference remains necessary before asserting deployment superiority.

Within these limits, one logical programmable p-bit per learned scalar supports the tested application-quality margins when finite controls, ranges, and temporal samples are chosen appropriately. The GPT-2 failure demonstrates why the assignment alone is insufficient: output-head activation structure interacts with range geometry. Layer and covariance diagnostics explain additional nonuniformity, while sensitivity-based allocation reduces reconstruction work. Strong spatial controls and finite-format counterexamples place concrete boundaries on the result. The defensible outcome is a reproducible map of feasible regimes and failure mechanisms, rather than universal BF16 replacement.

Code, data, and declarations. Package: `https://github.com/oramada/pbit1`. Complete research and manuscript: `https://github.com/oramada/pbit1-research` (public research repository). The original project code is Apache-2.0; third-party model and dataset licenses remain separate. This is independent research using Extropic software, without a claim of affiliation or endorsement. No physical Extropic hardware was used. See the companion supplement for evidence identifiers, exact protocols, and historical failures.

References

[1] Extropic. Z1T: Sparse transformer-like models for probabilistic hardware. Technical article, September 4, September 2026. URL https://extropic.ai/writing/z1t. Accessed November 6, 2026.

[2] Extropic. THRML: Thermodynamic hypergraphical models library. Software and documentation, 2026. URL https://github.com/extropic-ai/thrml/tree/2d54cd8404df85bb65703d7e8b4e84e48c1b72a9. Experiments pin commit 2d54cd8404df85bb65703d7e8b4e84e48c1b72a9.

[3] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.17323.

[4] David Hartmann and Michael Wand. Progressive stochastic binarization of deep networks. In *Workshop on Energy Efficient Machine Learning and Cognitive Computing, co-located with NeurIPS*, 2019. URL https://www.emc2-ai.org/assets/docs/neurips-19/emc2-neurips19-paper-23.pdf.

[5] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, 2024. URL https://arxiv.org/abs/2306.00978.

[6] Liquid AI. LFM3-1.7B model card and checkpoint. Hugging Face model repository, 2026. URL https://huggingface.co/LiquidAI/LFM3-1.7B/tree/ac2586a34f1a1c03329dbdcdefabd8b6478ddaa7. Revision ac2586a34f1a1c03329dbdcdefabd8b6478ddaa7.

[7] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, 2011. URL https://aclanthology.org/P11-1015/.

[8] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL https://arxiv.org/abs/1609.07843.

[9] Jorn W. T. Peters and Max Welling. Probabilistic binary neural networks, 2018. URL https://arxiv.org/abs/1809.03368.

[10] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[11] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter, 2019. URL https://arxiv.org/abs/1910.01108.

[12] John Shawe-Taylor and Jieyu Zhao. Generalisation of a class of continuous neural networks. In *Advances in Neural Information Processing Systems*, volume 8, 1995. URL https://proceedings.neurips.cc/paper/1995/file/6e7d2da6d3953058db75714ac400b584-Paper.pdf.